

Содержание

Введение	5
1 Исследовательский раздел	7
1.1 Анализ предметной области	7
1.2 Выбор программных средств	9
1.3 Вывод.....	12
2 Специальный раздел	14
2.1 Использование PostgreSQL для хранения информации	14
2.2 Интерфейс прототипа программного продукта.....	15
2.3 Вывод.....	18
3 Технологический раздел.....	20
3.1 Программный код основных функций	20
3.2 Защита информации	25
3.3 Вывод.....	26
Заключение	28
Перечень используемых информационных ресурсов	29

					РПМ.370000.000 ПЗ					
Изм.	Лист	№ докум.	Подпись	Дата	Разработка системы учёта проката спортивного инвентаря	Лит.		Лист	Листов	
Разраб.		Нестеренко Н.А.						4	30	
Провер.		Запорожец О.И.				ТИ (филиал) ДГТУ в г. Азове кафедра «ВТиП»				
Реценз.										
Н. Контр.										
Утверд.		Чумак И.В.								

Введение

В последние годы в России наблюдается значительный рост интереса к активному отдыху и любительскому спорту. Однако высокая стоимость качественного спортивного оборудования и сезонный характер многих видов спорта делают приобретение собственного инвентаря нецелесообразным для многих граждан. В связи с этим услуги пунктов проката становятся все более востребованными.

С ростом числа клиентов и расширением ассортимента предлагаемого инвентаря (от велосипедов и самокатов до горнолыжного снаряжения), традиционные методы учета, основанные на бумажных носителях, перестают обеспечивать необходимую скорость и точность обработки данных. Отсутствие автоматизации ведет к ряду проблем: возникновению очередей в пиковые часы, ошибкам при расчете стоимости услуг, трудностям в отслеживании возврата имущества и износа инвентаря. Для повышения эффективности работы подобных предприятий необходима разработка специализированной информационной системы. Именно решению этих задач посвящена данная курсовая работа.

Целью данной работы является проектирование и разработка базы данных для пункта проката спортивного инвентаря, а также создание программного интерфейса для автоматизации процессов выдачи, возврата и учета оборудования.

Для достижения поставленной цели необходимо выполнить следующие задачи:

1. рассмотреть и проанализировать предметную область пункта проката спортивного оборудования;
2. произвести обзор существующих программных решений и аналогов;

					РПМ.370000.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		5

3. обосновать выбор технологий и инструментальных средств (C#, PostgreSQL) для реализации программного продукта;
4. разработать структуру базы данных, обеспечивающую хранение информации об инвентаре, клиентах и операциях проката;
5. спроектировать и реализовать пользовательский интерфейс для эффективного взаимодействия с данными;
6. разработать алгоритмы автоматического расчета стоимости проката с учетом времени использования.

Структура работы:

1. в первом разделе проведен анализ бизнес-процессов пункта проката, описаны требования к системе и выполнен обзор аналогичного программного обеспечения;
2. во втором разделе рассмотрена архитектура базы данных, проведено проектирование таблиц и связей между ними, а также описан процесс разработки графического интерфейса;
3. в третьем разделе приведено описание программного кода ключевых функций (оформление проката, расчет выручки, управление базой клиентов) и представлены результаты тестирования системы.

1 Исследовательский раздел

1.1 Анализ предметной области

Объектом исследования является пункт проката спортивного инвентаря - динамично развивающаяся сфера услуг, ориентированная на популяризацию активного образа жизни и спорта. В современных условиях прокат выступает не только как сервис по предоставлению оборудования во временное пользование, но и как технологичное предприятие, требующее точного контроля материальных ценностей и времени.

В условиях современной экономики и высокой стоимости профессионального снаряжения (горных велосипедов, сноубордов, лыжной экипировки) потребители всё чаще отдают предпочтение аренде, а не покупке. Это позволяет пользователям пробовать различные виды спорта без значительных финансовых вложений, а предприятиям - извлекать прибыль за счет высокой оборачиваемости инвентаря. Однако специфика работы пункта проката сопряжена с необходимостью строгого учета каждой единицы имущества, мониторинга его технического состояния и контроля соблюдения сроков возврата.

Исторически прокат спортивного инвентаря развивался параллельно с ростом массового спорта. Если в середине XX века учет велся исключительно в бумажных журналах, то с появлением персональных компьютеров в 1980–90-х годах начались первые попытки автоматизации через простые табличные редакторы. На сегодняшний день развитие технологий позволяет внедрять системы, способные в реальном времени отслеживать статус каждой единицы оборудования и автоматически рассчитывать стоимость услуг.

Тема данной курсовой работы - проектирование и разработка базы данных для пункта проката спортивного инвентаря, а также создание прототипа информационной системы.

					РПМ.370000.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		7

1. с учётом бизнес-процессов предприятия, в проектируемой информационной системе должны быть предусмотрены следующие аспекты:
Учет инвентаря: ведение реестра доступного оборудования с фиксацией его характеристик, стоимости часа аренды и текущего статуса (в наличии, в прокате, в ремонте);

2. работа с клиентами: формирование базы данных клиентов с возможностью идентификации их статуса (включая проверку на наличие в «черном списке» за прошлые нарушения);

3. управление прокатами: автоматизация процесса оформления выдачи и приема инвентаря с фиксацией времени и расчетной стоимости.

Цель создания информационной системы: автоматизация процессов учета выдачи спортивного инвентаря, минимизация ошибок при расчетах, повышение прозрачности деятельности предприятия и обеспечение оперативного доступа к статистическим данным о выручке и популярности оборудования.

Задачи информационной системы:

1. регистрация новых единиц инвентаря и клиентов в едином хранилище данных;

2. оперативное изменение статусов объектов (например, перевод в статус «в ремонте» или «арендован»);

3. автоматический расчет итоговой стоимости услуг на основе установленных тарифов и длительности использования;

4. формирование отчетности по выручке за произвольный период времени для анализа финансовой эффективности предприятия;

5. идентификация клиентов, нарушивших правила проката, для ограничения их дальнейшего обслуживания.

1.2 Выбор программных средств

В качестве основной среды разработки для реализации программного продукта была выбрана Microsoft Visual Studio. Данная IDE является мощным инструментом профессиональной разработки, поддерживающим полный жизненный цикл создания ПО: от проектирования интерфейса до отладки и модульного тестирования.

Visual Studio предоставляет разработчику комплекс высокоэффективных инструментов:

1. интеллектуальный редактор кода: поддержка технологии IntelliSense обеспечивает контекстную подсказку имен методов, параметров и типов данных, что значительно ускоряет написание кода и снижает количество синтаксических ошибок;

2. визуальный конструктор: позволяет проектировать пользовательский интерфейс в режиме WYSIWYG, что упрощает компоновку элементов управления для системы учета проката;

3. механизмы отладки: встроенный дебаггер позволяет анализировать состояние переменных в режиме реального времени и отслеживать стек вызовов, что критически важно при реализации сложной логики расчета стоимости аренды;

4. интеграция с СУБД: встроенные обозреватели данных позволяют управлять схемой базы данных PostgreSQL непосредственно из среды разработки.

Основным языком разработки в рамках данной работы является C#. Это современный объектно-ориентированный язык с сильной типизацией, работающий на платформе .NET. Он сочетает в себе мощность C++ и простоту Java, предоставляя обширные библиотеки для работы с графикой, сетями и базами данных.

					РПМ.370000.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		9

Пример объектно-ориентированного подхода на языке C#, используемого при разработке системы, представлен в листинге 1.

Листинг 1 – Описание модели объекта инвентаря

```
public class SportInventory
{
    public int Id { get; set; }
    public string Name { get; set; }
    public decimal HourlyRate { get; set; }
    public string Status { get; set; }

    // Метод для расчета предварительной стоимости
    public decimal CalculatePrice(int hours)
    {
        return HourlyRate * hours;
    }
}
```

Язык C# позволяет эффективно обрабатывать коллекции данных и выполнять сложные вычисления. Пример реализации логики фильтрации данных представлен в листинге 2.

Листинг 2 – Пример обработки данных с использованием LINQ

```
var availableItems = allInventory
    .Where(item => item.Status == "В наличии")
    .OrderBy(item => item.Name)
.ToList();
```

Для взаимодействия с базой данных PostgreSQL в Visual Studio используется библиотека Npgsql, которая позволяет выполнять SQL-запросы непосредственно из кода приложения. Пример выполнения команды представлен в листинге 3.

Листинг 3 – Взаимодействие с базой данных через ADO.NET

```
using (var cmd = new NpgsqlCommand("SELECT SUM(total_price)
FROM rentals", connection))
{
    var result = cmd.ExecuteScalar();
    Console.WriteLine($"Общая выручка: {result}");
}
```

Или же как в пример использование как легких так и сложных цикловых операций, пример кода с использованием цикла будет предоставлен на листинге 4.

Листинг 4 – Использование простейшего цикла

```
using System;
class Program
{
    static void Main()
    {
        for (int i = 1; i <= 5; i++)
        {
            Console.WriteLine(i);
        }
    }
}
```

Благодаря своей гибкости и поддержке современных стандартов разработки, Microsoft Visual Studio является оптимальным выбором для создания надежной и масштабируемой системы учета. Логотип среды разработки представлен на рисунке 1.



Рисунок 1 – Логотип Visual Studio

1.3 Вывод

В ходе выполнения исследовательского раздела был проведен комплексный анализ предметной области - деятельности пункта проката спортивного инвентаря. Были выявлены ключевые бизнес-процессы, такие как учет оборудования, регистрация клиентов и автоматизация финансовых расчетов, требующие внедрения специализированного программного обеспечения.

Сравнительный анализ существующих аналогов показал, что готовые решения часто обладают избыточным функционалом или требуют значительных затрат на лицензирование и адаптацию под конкретные нужды малого предприятия. В связи с этим была обоснована целесообразность разработки собственного программного продукта, максимально ориентированного на поставленные задачи.

На основе проведенного исследования был сформирован стек технологий: в качестве среды разработки выбрана Microsoft Visual Studio, языком реализации определен объектно-ориентированный C#, а для хранения данных - реляционная СУБД PostgreSQL. Данный выбор позволит создать

надежное, масштабируемое приложение с интуитивно понятным интерфейсом и высокой скоростью обработки транзакций.

2 Специальный раздел

2.1 Использование PostgreSQL для хранения информации

В рамках данной курсовой работы в качестве системы управления базами данных (СУБД) была выбрана PostgreSQL. Это мощная объектно-реляционная система с открытым исходным кодом, которая отличается высокой надежностью, поддержкой сложных запросов и соответствием стандартам SQL.

Данные в системе организованы в виде таблиц, где каждая строка представляет собой отдельную запись, а столбцы - атрибуты объекта. Пример структуры хранения данных о совершенных прокатах представлен в листинге 5.

Листинг 5 – Пример структуры таблицы

```
CREATE TABLE rentals (  
    rental_id SERIAL PRIMARY KEY,  
    client_id INTEGER REFERENCES clients(client_id),  
    inventory_id INTEGER REFERENCES inventory(id),  
    start_time TIMESTAMP NOT NULL,  
    end_time_planned TIMESTAMP,  
    total_price DECIMAL(10, 2)  
);
```

Работа с данными в информационной системе «Прокат спортивного инвентаря» осуществляется через клиентское приложение. Все операции по добавлению, редактированию или удалению записей выполняются авторизованным сотрудником. Пример визуального представления данных из базы в интерфейсе программы представлен на рисунке 3.

Вот как хранится информация в формате данной работы на основе таблицы, показано на рисунке 2.

					РПМ.370000.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		14

	item_id	rental_id	client_id	start_time	end_time_planned	total_price
▶	2	2	2	02.05.2026 14:00	02.05.2026 17:00	
	3	1	1	01.05.2026 10:00	01.05.2026 12:00	900,00
	3	4	3	03.05.2026 15:08	03.05.2026 19:08	1600,00
	1	5	7	03.05.2026 15:09	03.05.2026 17:09	1000,00
	4	6	7	03.05.2026 15:24	03.05.2026 16:24	150,00

Рисунок 2 – Пример отображения данных таблицы проката в интерфейсе

Данные упорядочены по столбцам, что позволяет сотруднику быстро проводить поиск по ID клиента, названию инвентаря или дате выдачи. Использование параметров в SQL-запросах на стороне C# обеспечивает защиту от инъекций и стабильность работы приложения.

2.2 Интерфейс прототипа программного продукта

Проектирование интерфейса информационной системы основывалось на принципах обеспечения оперативной работы сотрудника пункта проката. Основное внимание было уделено логичному расположению элементов управления, позволяющему быстро переключаться между учетом инвентаря, списком клиентов и аналитическими данными.

Работа с системой начинается с окна авторизации. Доступ к функциям базы данных предоставляется только после ввода корректного логина и пароля, что необходимо для защиты коммерческой информации и персональных данных клиентов. Форма авторизации представлена на рисунке 3.

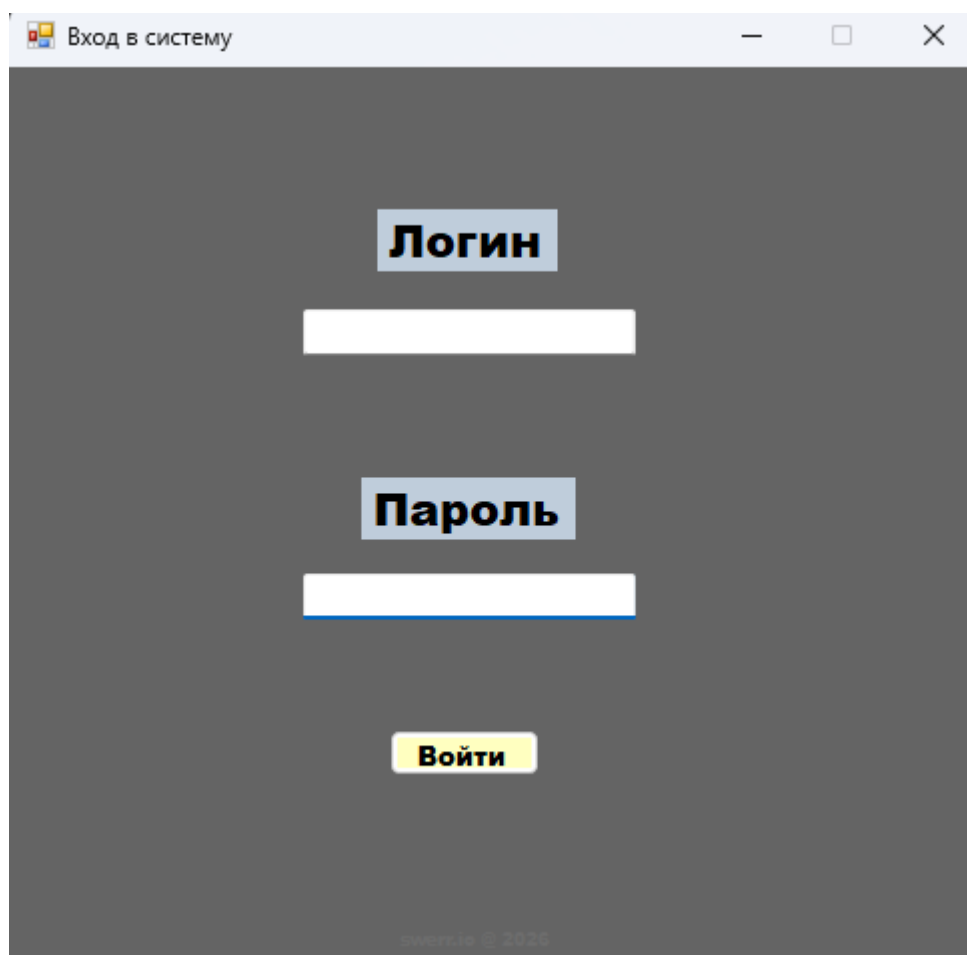


Рисунок 3 – Форма входа в систему

После прохождения авторизации открывается Главная форма программы, которая является основным рабочим пространством сотрудника. Интерфейс спроектирован по модульному принципу, где за каждый бизнес-процесс отвечает отдельный функциональный блок. Вид главной формы представлен на рисунке 4.

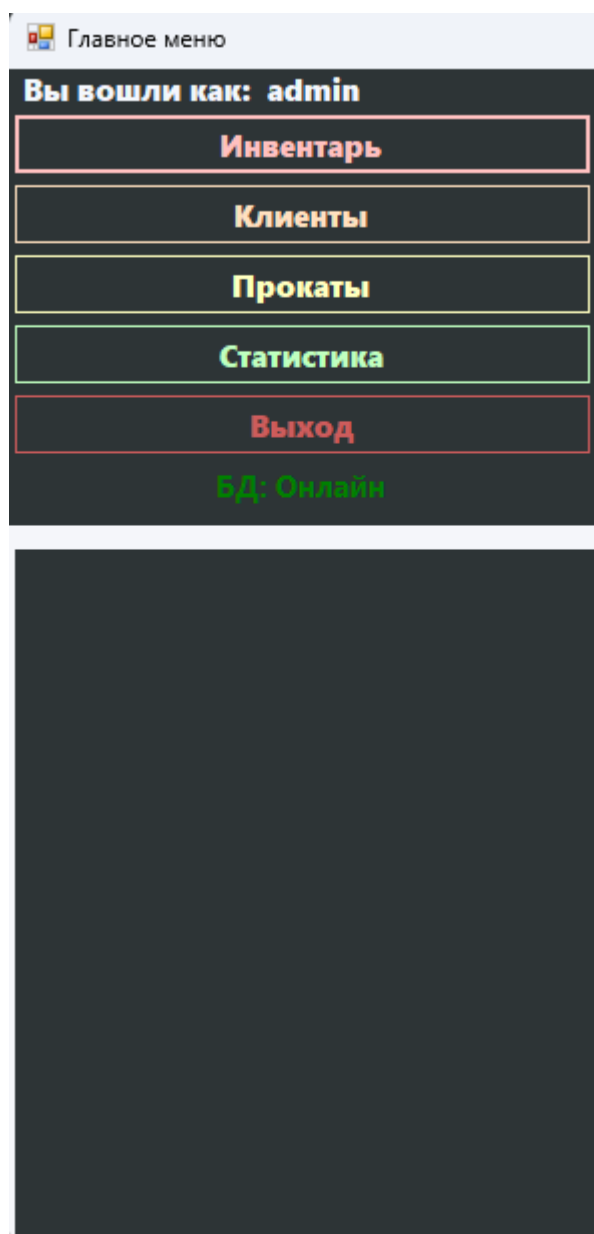


Рисунок 4 – Главная форма управления системой проката

На главной форме расположены навигационные элементы (кнопки), обеспечивающие доступ к следующим разделам системы:

1. учет инвентаря: вызов формы для просмотра всего списка спортивного оборудования, изменения его текущего статуса и добавления новых единиц;
2. база клиентов: работа с персональными данными посетителей, включая возможность проверки клиента на наличие в «черном списке»;

3. управление прокатами: регистрация выдачи инвентаря и оформление возврата с фиксацией времени;

4. статистика и выручка: модуль для расчета финансовых показателей за выбранный период времени, реализованный с помощью агрегатных функций SQL.

Для работы с данными используется элемент управления DataGridView, который позволяет отображать содержимое таблиц базы данных PostgreSQL в удобном табличном виде. Сотрудник имеет возможность не только просматривать записи, но и выполнять операции добавления, редактирования и удаления информации непосредственно через интерфейс приложения.

Пример окна для работы с конкретной таблицей списком инвентаря представлен на рисунке 5.

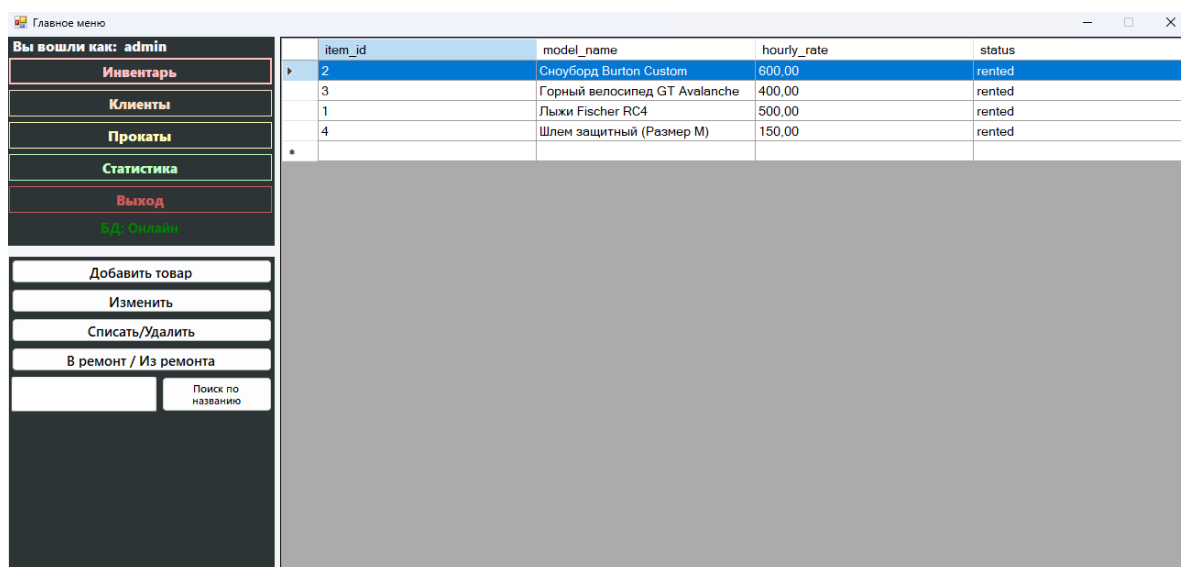


Рисунок 5 – Окно управления записями базы данных

2.3 Вывод

Во втором разделе курсовой работы была спроектирована архитектура информационной системы, определена структура базы данных в среде PostgreSQL и разработан графический интерфейс пользователя на языке C#.

В ходе проектирования были решены следующие задачи:

1. определен состав таблиц базы данных и установлены логические связи между ними, обеспечивающие целостность данных о прокате;
2. реализован программный механизм взаимодействия приложения с СУБД, позволяющий выполнять оперативный поиск, добавление и редактирование записей;
3. разработан интуитивно понятный пользовательский интерфейс для сотрудника предприятия, включающий формы авторизации, управления инвентарем и работы с клиентской базой;
4. реализованы алгоритмы автоматизации бизнес-процессов, таких как расчет итоговой выручки за отчетный период и контроль статусов оборудования.

Проведенная работа позволила создать функциональный прототип системы, полностью готовый к эксплуатации в условиях реального пункта проката спортивного инвентаря.

3 Технологический раздел

3.1 Программный код основных функций

Для обеспечения безопасности данных доступ к управлению базой данных осуществляется через форму авторизации. Код проверки полномочий сотрудника представлен в листинге 6.

Листинг 6 – Проверка полномочий при входе

```
private void btnLogin_Click(object sender, EventArgs e)
{
    string login = txtLogin.Text.Trim();
    string password = txtPassword.Text.Trim();

    if (string.IsNullOrEmpty(login) ||
string.IsNullOrEmpty(password))
    {
        MessageBox.Show("Пожалуйста, введите логин и пароль",
"Внимание");
        return;
    }

    using (var conn = new NpgsqlConnection(connString))
    {
        try
        {
            conn.Open();
            // Используется параметризованный запрос для
защиты от SQL-инъекций
            string sql = "SELECT full_name, role FROM employees WHERE
login = @login AND password_hash = crypt(@pass, password_hash)";

            using (var cmd = new NpgsqlCommand(sql, conn))
```

```

        {
            cmd.Parameters.AddWithValue("login", login);
            cmd.Parameters.AddWithValue("pass",
password);

            using (var reader = cmd.ExecuteReader())
            {
                if (reader.Read())
                {
                    string fullName =
reader.GetString(0);

                    this.Hide();
                    MainForm mainApp = new MainForm();
                    mainApp.FormClosed += (s, args) =>
this.Close();

                    mainApp.Show();
                }
                else
                {
                    MessageBox.Show("Неверный логин или
пароль", "Ошибка авторизации");
                }
            }
        }
    }
    catch (Exception ex)
    {
        MessageBox.Show("Ошибка работы с базой данных: "
+ ex.Message);
    }
}
}

```

Основная логика приложения сосредоточена в классе MainForm. Программа использует NpgsqlDataAdapter и DataTable для отображения данных в элементе DataGridView с поддержкой автоматического обновления. Код приведен в листинге 7.

Листинг 7 – Метод сохранения изменений в базу данных

```
private void SaveChanges()
{
    if (dt == null) return;
    try
    {
        using (var conn = new NpgsqlConnection(connString))
        {
            conn.Open();
            if (adapter != null)
            {
                NpgsqlCommandBuilder builder = new
NpgsqlCommandBuilder(adapter);
                adapter.Update(dt);          // Синхронизация
DataTable с БД
                RefreshCurrentTab();
                MessageBox.Show("Данные успешно сохранены!",
"Успех");
            }
        }
    }
    catch (Exception ex)
    {
        MessageBox.Show("Ошибка сохранения: " + ex.Message);
    }
}
```

Наиболее сложным узлом является форма оформления нового проката (RentForm). При сохранении заказа используется механизм транзакций для обеспечения целостности: одновременно создается запись в таблице rentals и изменяется статус оборудования в таблице inventory. Код приведен в листинге 8.

Листинг 8 – Оформление проката с использованием транзакции

```
private void BtnSave_Click(object sender, EventArgs e)
{
    using (var conn = new NpgsqlConnection(_connString))
    {
        conn.Open();
        var trans = conn.BeginTransaction(); // Начало
транзакции
        try
        {
            // Создание записи о прокате
            var cmd = new NpgsqlCommand(@"INSERT INTO rentals
(client_id, item_id, start_time, end_time_planned, total_price)
                                VALUES (@c, @i,
NOW(), NOW() + interval '1 hour' * @h, @p)", conn);
            cmd.Parameters.AddWithValue("c", client.Id);
            cmd.Parameters.AddWithValue("i", item.Id);
            cmd.Parameters.AddWithValue("h",
(int)numHours.Value);
            cmd.Parameters.AddWithValue("p", total);
            cmd.ExecuteNonQuery();

            // Обновление статуса товара
            var cmdUpdate = new NpgsqlCommand("UPDATE
inventory SET status = 'rented' WHERE item_id = @i", conn);
            cmdUpdate.Parameters.AddWithValue("i", item.Id);
            cmdUpdate.ExecuteNonQuery();
        }
    }
}
```

```

        trans.Commit();
        MessageBox.Show("Прокат успешно оформлен!");
    }
    catch (Exception ex)
    {
        trans.Rollback();
        MessageBox.Show("Ошибка: " + ex.Message);
    }
}
}

```

Система позволяет проводить оперативный анализ деятельности пункта проката, включая расчет выручки за произвольный период и экспорт текущих таблиц в формат CSV для последующей обработки. Код показан в листинге 9.

Листинг 9 – Расчет выручки за период средствами SQL

```

private void btnStatsRevenue_Click(object sender, EventArgs
e)
{
    string start = Prompt.ShowDialog("Начало (ГГГГ-ММ-ДД):",
"Выручка");
    string end = Prompt.ShowDialog("Конец (ГГГГ-ММ-ДД):",
"Выручка");

    if (!string.IsNullOrEmpty(start) &&
!string.IsNullOrEmpty(end))
    {
        using (var conn = new NpgsqlConnection(connString))
        {
            conn.Open();
            string sql = @"SELECT SUM(total_price) FROM
rentals
WHERE start_time::date >=
@start::date

```

					РПМ.370000.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		24

```

                                AND      start_time::date      <=
@end::date";

        var cmd = new NpgsqlCommand(sql, conn);
        cmd.Parameters.AddWithValue("start", start);
        cmd.Parameters.AddWithValue("end", end);

        var res = cmd.ExecuteScalar();
        decimal totalRevenue = (res == DBNull.Value) ? 0
: Convert.ToDecimal(res);

        MessageBox.Show($"Выручка      за      период:
{totalRevenue} руб.");
    }
}
}

```

3.2 Защита информации

В соответствии с требованиями к защите персональных данных сотрудников, в системе реализовано хэширование паролей с использованием расширения pgcrypto для СУБД PostgreSQL. Данный подход исключает хранение паролей в открытом виде, что предотвращает их компрометацию даже в случае получения злоумышленником доступа к файлам базы данных.

Для проверки подлинности пользователя используется функция crypt(). В программном коде на стороне C# это реализовано через передачу введенного пользователем пароля в SQL-запрос, где база данных сравнивает его с сохраненным хэшем.

Без использования дополнительных утилит доступны следующие технологии хэширования:

1. md5 (MD5) – 128-битный алгоритм хеширования. Предназначен для создания «отпечатков» или дайджестов сообщения произвольной длины и последующей проверки их подлинности. Широко применялся для проверки целостности информации и хранения хэшей паролей.

					<i>РПМ.370000.000 ПЗ</i>	Лист
Изм.	Лист	№ докум.	Подпись	Дата		25

2. des (DES) – алгоритм для симметричного шифрования, разработанный фирмой IBM и утверждённый правительством США в 1977 году как официальный стандарт. Алгоритм использует комбинацию нелинейных (S-блоки) и линейных (перестановки E, IP, IP-1) преобразований.

3. xdes (Extended DES) – улучшенная осовремененная версия алгоритма des.

4. bf (Blowfish) – криптографический алгоритм, реализующий блочное симметричное шифрование с переменной длиной ключа. Выполнен на простых и быстрых операциях: XOR, подстановка, сложение. Является незапатентованным и свободно распространяемым.

Пример хэширования представлен листинге 10.

Листинг 10 – Шифрование паролей в PostgreSQL

```
UPDATE employees SET password_hash = crypt(password_hash,  
gen_salt('bf'));
```

Для проверки подлинности пользователя используется функция crypt(). В программном коде на стороне C# это реализовано через передачу введенного пользователем пароля в SQL-запрос, где база данных сравнивает его с сохраненным хэшем. Код предоставлен в листинге 11.

Листинг 11 – Метод авторизации с применением хэширования на стороне БД

```
string sql = "SELECT full_name, role FROM employees " + "WHERE  
login = @login AND password_hash = crypt(@pass, password_hash)";
```

3.3 Вывод

Применение алгоритма хэширования через библиотеку pgcrypto позволяет организовать надежную систему аутентификации. Соль генерируется автоматически функцией gen_salt(), что делает хэш уникальным

даже для одинаковых паролей, значительно усложняя задачу подбора пароля по словарям.

Заключение

Курсовая работа является необходимым элементом учебного процесса, так как её основная цель — создание законченного программного продукта и составление полного комплекта документации, что позволяет студенту закрепить навыки квалифицированного специалиста.

Во время написания работы была спроектирована и разработана база данных для автоматизации учета пункта проката спортивного инвентаря, полностью удовлетворяющая поставленным требованиям. В ходе выполнения проекта была изучена предметная область, проведен анализ программных средств (C#, PostgreSQL), разработана инфологическая и даталогическая модели данных.

Особое внимание было уделено защите персональных данных сотрудников: реализован механизм хэширования паролей с использованием расширения pgcrypto, что гарантирует безопасность системы. Разработанный прототип приложения позволяет эффективно управлять клиентской базой, инвентарем и транзакциями проката. Таким образом, цель курсовой работы достигнута, все задачи выполнены в полном объеме.

					РПМ.370000.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		28

Перечень используемых информационных ресурсов

1. Васильев, А. Н. Программирование на С# для начинающих. Основные сведения / А. Н. Васильев. — Москва: Эксмо, 2022. — 528 с.
2. Вагнер, Б. Эффективное программирование на С#. 50 способов улучшения кода / Б. Вагнер. — Москва: Вильямс, 2021. — 224 с.
3. Гринченко, Н. Н. Проектирование баз данных. Учебное пособие / Н. Н. Гринченко. — Москва: Горячая линия-Телеком, 2021. — 240 с.
4. Джеффри, Р. PostgreSQL 15. Изнутри / Р. Джеффри. — Москва: ДМК Пресс, 2023. — 650 с.
5. Исаев, Г. Н. Проектирование информационных систем: учебное пособие / Г. Н. Исаев. — Москва: Омега-Л, 2020. — 424 с.
6. Коннолли, Т. Базы данных. Проектирование, реализация и сопровождение. Теория и практика / Т. Коннолли, К. Бегг. — Москва: Вильямс, 2020. — 1440 с.
7. Культин, Н. Б. Основы программирования в Microsoft Visual C# / Н. Б. Культин. — Санкт-Петербург: БХВ-Петербург, 2022. — 288 с.
8. Лукин, В. Н. Введение в проектирование баз данных / В. Н. Лукин. — Москва: Вузовская книга, 2020. — 144 с.
9. Мартин, Р. Чистый код: создание, анализ и рефакторинг / Р. Мартин. — Санкт-Петербург: Питер, 2022. — 464 с.
10. Мюллер, Р. Д. Проектирование баз данных и UML / Р. Д. Мюллер. — Москва: Лори, 2021. — 420 с.
11. Пирогов, В. Ю. Информационные системы и базы данных: организация и проектирование / В. Ю. Пирогов. — Санкт-Петербург: БХВ-Петербург, 2021. — 528 с.
12. Рихтер, Д. CLR via C#. Программирование на платформе Microsoft .NET Framework 4.5 на языке C# / Д. Рихтер. — Санкт-Петербург: Питер, 2020. — 896 с.

13. Туманов, В. Е. Основы проектирования реляционных баз данных / В. Е. Туманов. — Москва: Интернет-Университет Информационных Технологий, 2021. — 420 с.

14. Фленов, М. Е. Библия С# / М. Е. Фленов. — Санкт-Петербург: БХВ-Петербург, 2023. — 560 с.

15. Шилдт, Г. С# 8.0. Полное руководство / Г. Шилдт. — Москва: Диалектика, 2021. — 1000 с.

					РПМ.370000.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		30